

# Model-Aided Deep Learning for Signal Processing with Ferroelectret-based Force Sensors

Master thesis in the department of Electrical Engineering and Information Technology by Hao Yuan

Date of submission: May 26, 2025

1. Review: Prof. Dr.-Ing. Marius Pesavento

2. Review: Lukas Schynol M.Sc.  
Darmstadt



TECHNISCHE  
UNIVERSITÄT  
DARMSTADT

Electrical Engineering and  
Information Technology  
Department

Nachrichtentechnische  
Systeme

---

### Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, <sup>Hao</sup>Vorname <sup>Yuan</sup>Nachname, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs.2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird.

Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

---

### English translation for information purposes only:

### Thesis Statement pursuant to § 22 paragraph 7 of APB TU Darmstadt

I herewith formally declare that I, <sup>Hao</sup>first name <sup>Yuan</sup>last name, have written the submitted thesis independently pursuant to § 22 paragraph 7 of APB TU Darmstadt without any outside support and using only the quoted literature and other sources. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I have clearly marked and separately listed in the text the literature used literally or in terms of content and all other sources I used for the preparation of this academic work. This also applies to sources or aids from the Internet.

This thesis has not been handed in or published before in the same or similar form.

I am aware, that in case of an attempt at deception based on plagiarism (§38 Abs. 2 APB), the thesis would be graded with 5,0 and counted as one failed examination attempt. The thesis may only be repeated once.

For a thesis of the Department of Architecture, the submitted electronic version corresponds to the presented model and the submitted architectural plans.

---

Datum / Date:

26.05.2025

Unterschrift/Signature:

Hao Yuan

---

# Acknowledgment

---

I would like to express my deepest gratitude to Prof. Dr. Pesavento for suggesting the topic of this research and also providing a great deal of help and advice during the research process. I am also grateful to Mr. Schynol M.Sc. for being my supervisor and for providing a wealth of useful advice and suggestions.

I would like to thank Dr. Chadda and Dr. Omar Ben Dali from MimoSense, they provided the sensor used in this study, and they provide a lot of data, the theoretical basis of the sensors, and experimental help. I would like to thank all the members of the MimoSense working group, as well as my alumnus, Mr. Wongodeo, for providing a great deal of help with the measurement process.

Thanks to the TU Darmstadt and the NTS group for providing the devices used for experiment and quiet and clean office space. Thanks also to all the members of the NTS group for their help.

I would like to further extend my sincere thanks to my friends and classmates for their unconditional support.

Grateful sentiments go to my girlfriend Yuting Yong, for her unwavering support and belief in me. I couldn't have disciplined myself to get so much done without your motivation.

Finally, I must thank my family who have always supported me, my parents and grandparents, who have selflessly supported me and encouraged me wherever I am.

---

# Abstract

---

This thesis focuses on how to estimate the value of force based on force sensor measurements.

3D-printed ferroelectret-based force sensors are new, cost-effective alternative to traditional load cells. They can measure two voltages at the same time, one obtained from the change in electrical resistance and the other from piezoelectric effect. The biggest challenge of force estimation is how to better use these two measurements to minimize error.

This thesis proposes two data-driven approaches to eliminate drift and hysteresis for printed ferroelectret-based force sensors. The first approach is based on a temporal convolutional neural network(TCN) and the second approach is deep learned Kalman filter for ferroelectret-based force sensor(DLKF-FFS).

For TCN model, the measured voltages are used as inputs to the neural network and the estimated force values are the outputs.

DLKF-FFS is a hybrid with partly physics-inspired and partly data-driven components. The model based on realistic physics is developed and used to design the basic Kalman filter. This Kalman filter was subsequently trained as part of a deep neural network. The measured voltages are used as the input of the neural network as well as the observation of this Kalman filter.

As a result of the experiments, it can be concluded that both approaches are capable of estimating the value of force with low error and able to eliminate both drift and hysteresis.

---

# Contents

---

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                       | <b>7</b>  |
| <b>2</b> | <b>Fundamentals</b>                                                       | <b>10</b> |
| 2.1      | Piezoelectric effect and the electrical resistance strain gauge . . . . . | 10        |
| 2.2      | Hysteresis and drift . . . . .                                            | 11        |
| 2.2.1    | Drift error . . . . .                                                     | 12        |
| 2.2.2    | Hysteresis error . . . . .                                                | 12        |
| <b>3</b> | <b>Measurement</b>                                                        | <b>14</b> |
| 3.1      | Data collection . . . . .                                                 | 14        |
| 3.1.1    | Reference measurement . . . . .                                           | 14        |
| 3.1.2    | Main measurements . . . . .                                               | 17        |
| 3.2      | Data preprocessing . . . . .                                              | 20        |
| 3.2.1    | Low pass filter . . . . .                                                 | 20        |
| 3.2.2    | Synchronization . . . . .                                                 | 20        |
| 3.2.3    | Down sample and interpolation . . . . .                                   | 21        |
| <b>4</b> | <b>Proposed Estimation Methods</b>                                        | <b>23</b> |
| 4.1      | Temporal convolutional network . . . . .                                  | 24        |
| 4.1.1    | Datasets . . . . .                                                        | 26        |
| 4.1.2    | Training and validation . . . . .                                         | 26        |
| 4.2      | Deep learned Kalman Filter . . . . .                                      | 28        |
| 4.2.1    | Model of Kalman Filter . . . . .                                          | 29        |
| 4.2.2    | Kalman Net . . . . .                                                      | 32        |
| 4.2.3    | Neural network layer structure . . . . .                                  | 33        |
| 4.2.4    | Training and validation . . . . .                                         | 34        |
| 4.3      | Real time implementation . . . . .                                        | 38        |



|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>5</b> | <b>Experimental Results</b>                           | <b>40</b> |
| 5.1      | Temporal Convolutional Neural Network (TCN) . . . . . | 40        |
| 5.2      | DLKF-FFS . . . . .                                    | 43        |
| 5.3      | Comparison . . . . .                                  | 46        |
| <b>6</b> | <b>Conclusion and Outlook</b>                         | <b>50</b> |

---

# 1 Introduction

---

Load cells are widely used in different measuring instruments such as laboratory balances, industrial scales, platform scales and universal testing machines, they are usually robust and accurate but expensive. Therefore, for small devices, low-cost force sensors are required that are accurate and sensitive to small forces. Some of them use Force Sensing Resistors (FSRs), also known in literature as piezoresistive sensors. FSRs suffer from significant hysteresis, which dramatically reduces the linearity during unloading forces. Moreover, considerable drift error is also exhibited by FSRs yielding substantial deviations of the reading during static forces [1]. Another alternative is Electrical Resistance Strain Gauges (ERSGs) which operate on the principle of an electrical conductor undergoing a change in electrical resistance due to geometry change. However, ERSGs also face similar hysteresis and drift problems as FSRs.

In response to these problems faced by FSRs and ERSGs, Ben Dali proposes a new printed force sensor based on his work [2]. The sensors are using Ferroelectrets and exhibit both piezoelectric effects and resistance change when being pressed. This allows the sensor to measure two voltages simultaneously, one accurate for static force values and the other sensitive to small changes in force. The two readings need to be processed appropriately in order to obtain an accurate force. This thesis discuss different methods on this signal processing problem.

Deep learning has become very popular in recent years in many fields. For example, in Computer vision, since the success of AlexNet [3], deep learning has been widely used for image classification. In natural language processing, the Transformer [4] revolutionized sequence modeling, leading to powerful models like BERT [5]. Deep learning has also impacted scientific research, with AlphaFold [6] accurately predicting protein structures, and AlphaGo [7] defeating human champions in the game of Go. These breakthroughs demonstrate deep learning's versatility and transformative power across domains.

---

One of the well known approaches of deep learning is Convolutional Neural Network(CNN). In academic research, CNNs have become indispensable tools for extracting complex patterns from structured data, particularly images across disciplines. In medical imaging, CNNs power state-of-the-art systems for disease diagnosis, enabling classification, segmentation, localization, and detection of pathologies in modalities ranging from MRI to histopathology [8], [9]. In astronomy, they can analyze enormous survey datasets, categorize stellar spectra, and identify rare phenomena [10], [11]. Moreover, CNNs are increasingly proved to be applicable to the field of signal processing [12], [13], [14].

Mohseni et al.'s paper discusses the use of convolutional neural networks to predict ground reaction force based on 3D-printed ferroelectret-based sensors measurements [15]. Their experiments show that simple convolutional neural networks are feasible in processing this kind of data. However, since convolutional neural networks are non-causal, the goal of real-time measurement for real-time estimation cannot be achieved. In addition, convolutional neural networks are not very robust, require a large number of parameters and they are difficult to interpret.

The Kalman filter(KF) is a fundamental technique in signal processing. It was first described and developed in 1950s and is used in a large number of different applications. It obtains the Minimum Mean-Square-Error (MMSE) estimation of a system state by recursively drawing observed samples and updating the estimate. Across disciplines, it underpins guidance and navigation in aerospace, sensor fusion in robotics, time-series analysis in economics, image restoration in medical diagnostics, and state estimation in industrial processes [16] [17].

Tran Phuong et al. exploited a multi-sensor data fusion by Kalman filter and states that the Kalman filter using multisensor can provide an accurate velocity estimation and the effective noise suppression [18]. Their study demonstrated that using Kalman filtering for multi-sensor fusion provides the enhanced force sensing performance and the effective noise reduction.

The seminar work in [19] demonstrates that Kalman filter can be used as part of a neural network. Monga [20] pointed out that a neural network is essentially a parameter optimization problem, so it is possible to put Kalman filter as a part of neural network and train it, thereby interpreting the observation of the KF as the deep network input and the state estimate as the output. The authors of [21] proves the feasibility of elimination of variable-frequency disturbances using deep learning based Kalman filter for force sensation.



---

Combining the KF with deep network components offers advantages such as fully interpretable and fully adaptive [22]. Based on these advantages, in sensor signal processing, pre-trained models that can be adapted to multiple sensors are available, and parameter tuning based on usage scenarios becomes possible.

The main idea of this thesis is to discuss the feasibility of AI-aided Kalman Filter for eliminating noise, hysteresis and drift in Ferroelectret-based force sensors.

This thesis discusses the feasibility of deep learned Kalman filter to directly process two noise-laden signals in the time domain and estimating unknown signals. The advantages and disadvantages of the time convolution neural network and deep learned Kalman filter for processing one-dimensional two-channel time-dependent signals are compared. In addition, the possibility of using piezoelectric effects and change in resistance to reduce hysteresis and drift in force sensors is discussed.

This thesis is based on the concept of deep unrolling [20] coupled with ai-aided Kalman filtering [22], and fills the blank of using deep learned Kalman filter to process two time-domain observations.

Chapter 2 of this thesis briefly describes how these Ferroelectret-based printed force sensors work. Chapter 3 describes the measurement method, the experimental design of the measurement data, and the measurement process. Chapter 4 describes two signal processing methods for the collected data, temporal convolutional neural network(TCN) and deep learned Kalman filter(DLKF-FFS). Chapter 5 explains the results and comparative analyses.

---

## 2 Fundamentals

---

---

### 2.1 Piezoelectric effect and the electrical resistance strain gauge

---

The shape of the sensor investigated in this thesis is shown in Fig. 2.1. The sensor are developed by MimoSense, and are based on the research in [2] by Ben Dali. His dissertation elucidates the manufacturing side of a new Ferroelectretret-based sensor. The FEP films and TPU spacer are sandwiched in an aluminium holder together constructed the sensor.

The sensor exhibits a different piezoelectric response at different pressure levels. The piezoelectric effect is the phenomenon in which a piezoelectric material produces a charge  $Q$  and a voltage  $v$  in response to an external force. The resulting voltage is given as

$$v_o = \frac{f_d d_{33}}{c_m} \quad (2.1)$$

where  $f_d$  is the external force,  $d_{33}$  is the piezoelectric coefficient and  $c_m$  is the capacitance.

By designing the circuit with  $R_1, R_2$  as the measurement resistors, the measured voltage external is

$$v_d = \left(1 + \frac{R_2}{R_1}\right) v_o = \left(1 + \frac{R_2}{R_1}\right) \frac{d_{33}}{c_m} f_d. \quad (2.2)$$

The electrical resistance strain gauge is a resistance element which changes resistance when subject to strain. In contrast to the piezoelectric effect, the change occurs in



Figure 2.1: Printed force sensors used in the experiment

electrical resistance. The resistance change  $\Delta R$  in the material is mostly due to the change of geometry resulting from applied mechanical stress as

$$\Delta R = \frac{\rho \Delta l}{A}, \quad (2.3)$$

where we assume that the area  $A$  and the electrical resistivity  $\rho$  stay constant, then  $\Delta R$  only depends on the change of length  $\Delta l$ . By designing the Wheatstone bridge circuit,  $\Delta R$  can be expressed by the following equation

$$v_s = v_0 \frac{\Delta R}{4R_0} \quad (2.4)$$

where  $v_s$  denotes the measured value,  $v_0$  denotes the constant input voltage and  $R_0$  serves as a protective resistor. Therefore, the measured voltage becomes

$$v_s = \frac{\rho v_0}{4R_0 A} \Delta l. \quad (2.5)$$

---

## 2.2 Hysteresis and drift

---

According to [23], hysteresis is a nonlinearity that the output displacement depends on a combination of the currently applied voltage as well as on some past values of the

---

applied voltage. It is more pronounced over large measurement ranges. On the other hand, drift can be seen as a slow creep in the measurement after responding to a sudden change in the input voltage. Drift is more prominent in slow and static parts. Fig. 2.2 show how drift and hysteresis behave over a segment of measurements.

### 2.2.1 Drift error

Let  $r_i$  be the reading immediately upon the application of force. Let  $r_f$  be final resistance reading after a specified time, so it can be considered that the drift phenomenon during this time causes an error (DER) of

$$\text{DER} = 100\% \frac{(r_f - r_i)}{r_i}. \quad (2.6)$$

### 2.2.2 Hysteresis error

Let  $r_l$  and  $r_u$  be the reading that correspond to the same applied force, but they are the readings acquired when loading and unloading forces are applied, respectively. The values of them are collected when the difference is maximal. Let  $r_m$  be the reading for the maximal applied force during the procedure, then the hysteresis error (HER) can be defined as

$$\text{HER} = 100\% \frac{(r_u - r_l)}{r_m}. \quad (2.7)$$

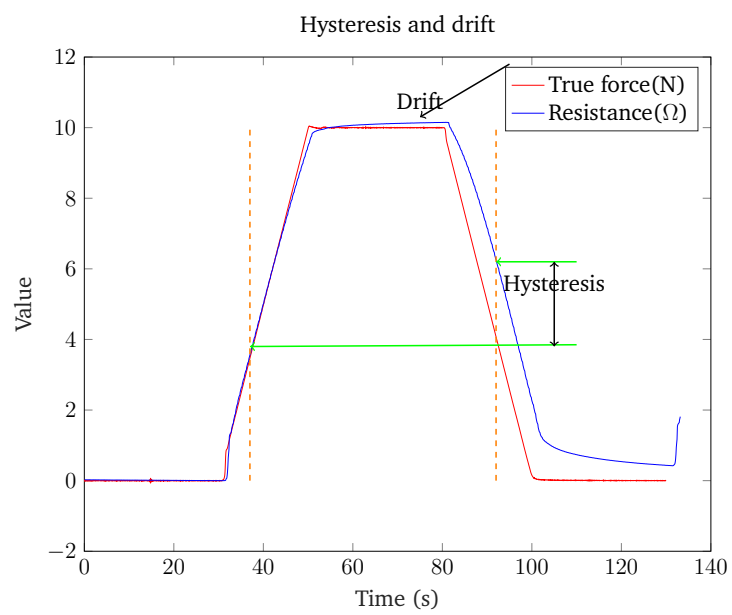


Figure 2.2: Hysteresis and drift error

---

## 3 Measurement

---

---

### 3.1 Data collection

---

We conducted several experimental runs to collect data in order to obtain enough data for our studies. A *Hegewald Peschke* universal testing machine (Fig. 3.1) was used, which is programmable and presses objects according to the programmed force transient. It records the current displacement and pressure in real time. This pressure reading was considered to be an accurate and error-free reference value in the experiments.

#### 3.1.1 Reference measurement

For our initial reference measurements, we design two force transients. The first measurement is designed as shown in Fig. 3.2, where the pressure increased at a rate of 2.5 N/s, pauses for a few seconds after reaching 10 N, and then returns to 0 at the same rate, and so on for several repetitions.

The second measurement process is taken as shown in Fig. 3.3, where the press stayed longer at both 10N and 5N to evaluate the performance at constant force.

In the experiment, the test machine obtained the reference value with a sampling frequency of 50 Hz.

In addition, a *Keithley* DMM6500 multimeter is used in the experiment to record the change in resistance under different force. Without any pre-processing of this data, the hysteresis and drift is quantitatively analyzed.

Since the DMM is considered to be very accurate, this set of data is considered to be the hysteresis and drift characteristics of the sensor itself when not processed in any way.



Figure 3.1: 'Hegewald Peschke' Universal testing machine

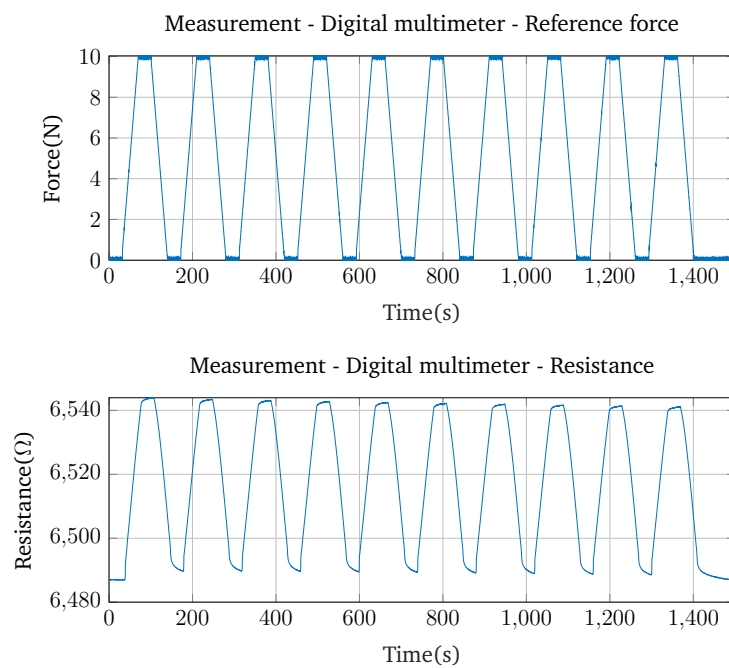


Figure 3.2: Measurement data of DMM

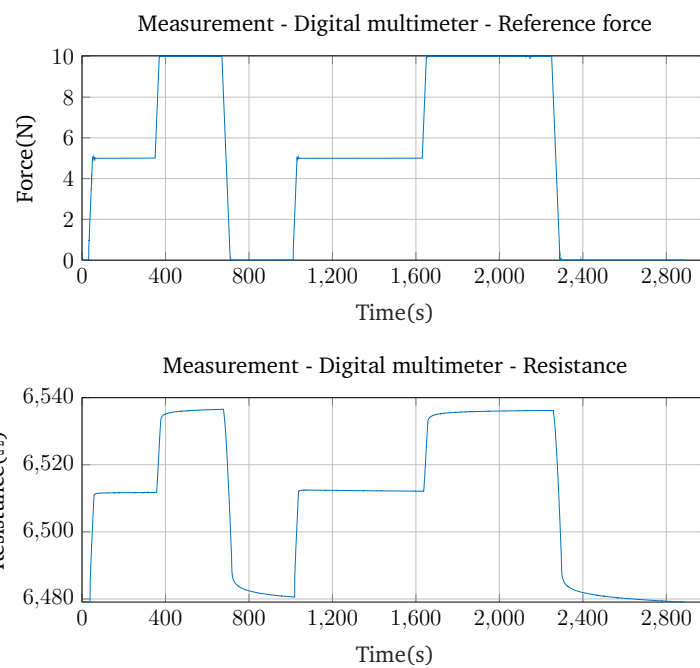


Figure 3.3: Measurement data of DMM



---

---

|         | HER    | DER(per second) |
|---------|--------|-----------------|
| Maximum | 17.36% | 0.99‰           |
| Average | 14.26% | 0.88‰           |

Table 3.1: Reference Hysteresis and Drift error

Accordin to Eq. (2.6) and Eq. (2.7), hysteresis and drift error are calculated, which is shown in Tab. 3.1.

### 3.1.2 Main measurements

After that, instead of using the DMM, the measurement tool provided by MimoSense is used to make the measurements. The raw data obtained is  $v_d$  from Eq. (2.2) and  $v_s$  from Eq. (2.5). These two data were used for subsequent processing. In the experiment, the sensor's data collector samples at 500 Hz.

At this stage, several more stress test transients are designed. These include transients resembling impulse responses, fast oscillatory transients based on 5N and step function waveforms with a step of 2.5N. To ensure consistency in the values of  $d_{33}$ , the maximum force for the experiment is limited to 10N.

In addition, the duration of one experiment is as long as possible in order to include a large number of different waveforms, as shown in Fig. 3.4, in the experimental data, the duration is 1400s and contains 8 different transients, each of which is repeated at least 4 times.

Furthermore, waveforms with different force gradients are designed to simulate different usage scenarios. For this purpose, another experiment was designed in which the waveforms changed at a much slower rate, as shown in Fig. 3.5.

All these data can enrich the training set as well as test the generalisation of the estimation methods, in particular data-driven deep learning approaches. The performance of these data after processing, including the degree of hysteresis, the degree of drift and the noise level are considered important indicators of the results.

In order to ensure that the inconsistency of different sensors will not affect the results of this experiment, the data all come from the same sensor named SDK002.

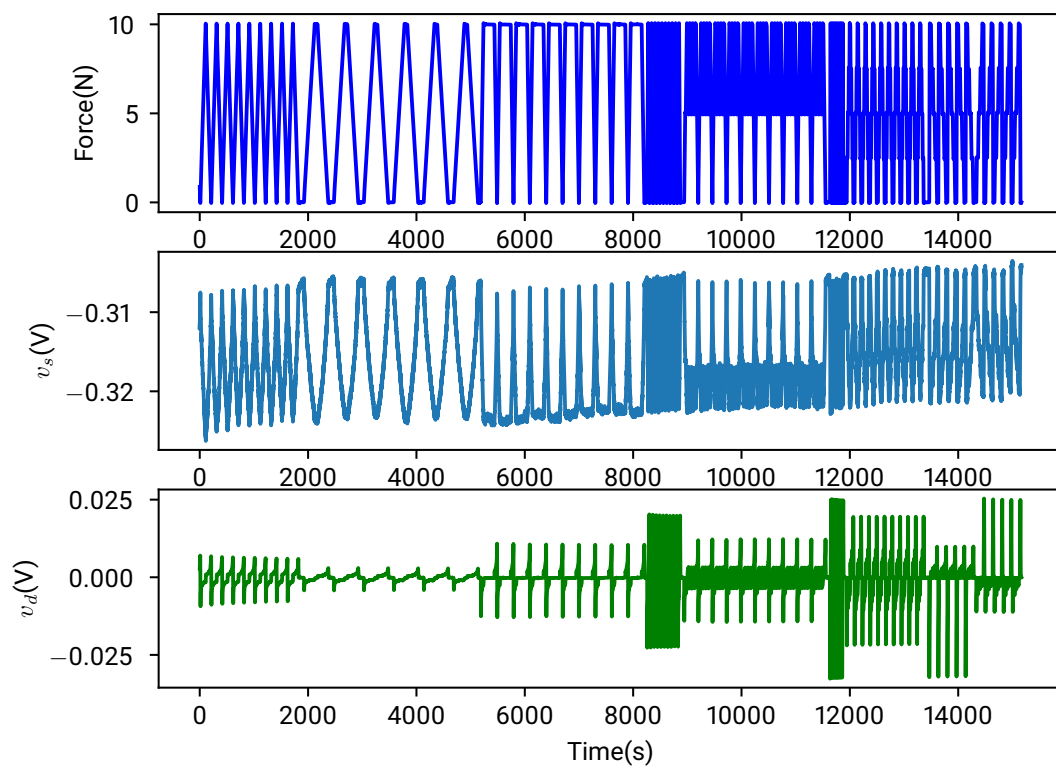


Figure 3.4: Long measurement data including different waves with quicker changes

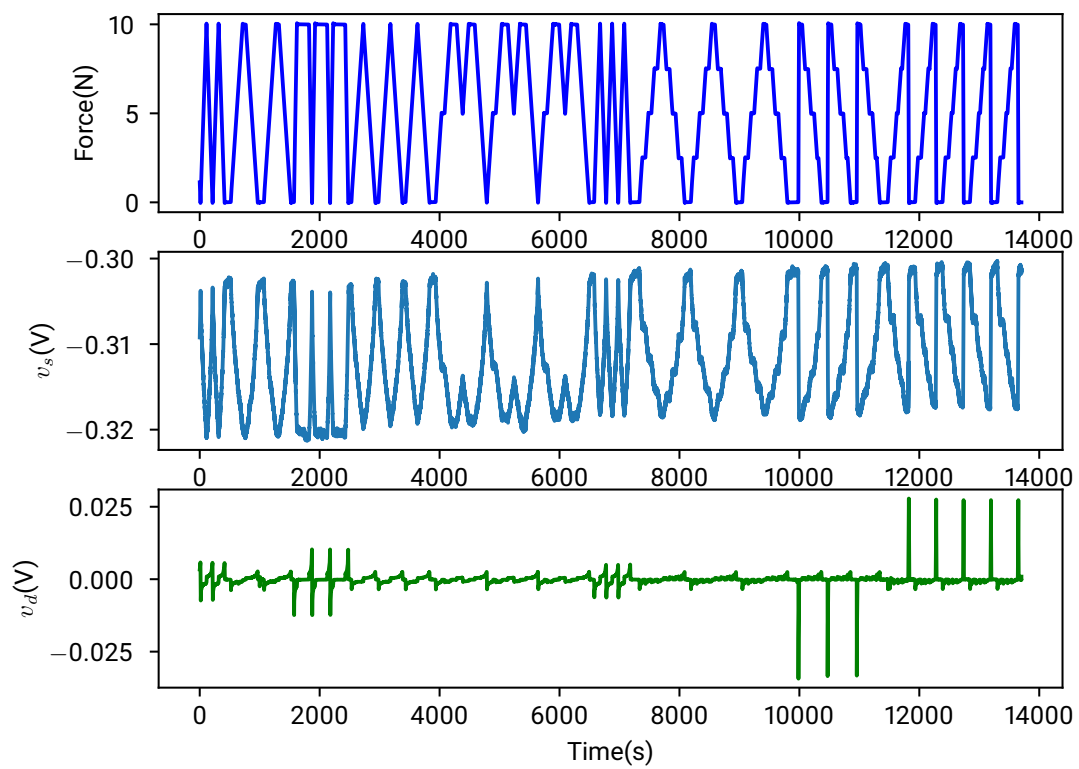


Figure 3.5: Long measurement data including different waves with slower changes

---

## 3.2 Data preprocessing

---

### 3.2.1 Low pass filter

It can be seen that the raw measurement data are noisy, both in  $v_s$  and  $v_d$ . In the preprocessing step, an eighth-order Butterworth low-pass filter with a normalized cutoff frequency of  $f_c$  as

$$f_c = 0.08 \frac{f_s}{2} \quad (3.1)$$

is employed to attenuate high-frequency noise while preserving the signal's low-frequency components. The Butterworth design was chosen for its maximally flat passband response, ensuring a ripple-free amplitude characteristic within the passband, and the 8th-order configuration provides a sufficiently steep transition band without introducing excessive group delay.

The predominant noise in our experimental setup arises from environmental sources, such as ambient electromagnetic interference, sensor thermal fluctuations, and mechanical vibrations which collectively approximate a Gaussian, broadband spectrum. Under the assumption of Gaussian white-noise statistics, the Butterworth filter's smooth attenuation profile effectively suppresses these stochastic disturbances without imposing spurious spectral artifacts. This combination of Gaussian-noise modeling, Butterworth design, and bidirectional filtering thus delivers robust high-frequency artifact rejection while preserving the integrity of time-domain features.

### 3.2.2 Synchronization

During the data collection process, the start button of the Universal testing machine and the voltage data collection button of the sensor are pressed manually, so the force reference value data and the  $v_s$ ,  $v_d$  measurements are always out of synchronization, with a time difference of about 3 to 10 seconds. Similarly, at the end of the experiment, which is also stopped manually, the end of the data is also out of synchronization, with a time difference of roughly 5 seconds.

The desynchronization is critical and it severely affects the results of the subsequent data processing. An incorrect synchronization step is fatal as it can make the experiment completely meaningless.

---

The sudden change in voltage due to the piezoelectric effect should occur at the same time as the sudden change in pressure applied to the sensor. Based on this principle, the algorithm on how to synchronize this problem is as follows. At time  $\tau$  that contains only the first sudden change of force, find the time point where the maximum in the derivatives of the piezo electric voltage and force, respectively, is located. The difference of samples  $G$  is estimated as

$$\hat{G} = \arg \max_{0 \leq s_1 \leq \tau} \frac{dv_d}{dt} - \arg \max_{0 \leq s_2 \leq \tau} \frac{df}{dt}. \quad (3.2)$$

The one that is delayed removes  $\hat{G}$  samples at the beginning of the sequence.

Next, the longer one is truncated at the end so that their lengths are equal, thus solving the problem of unsynchronised ending times.

### 3.2.3 Down sample and interpolation

Aligning signals on the same set of time instants is essential for the following analyses such as scaling and clipping operations.

Both up sampling and down sampling are used in the experiment.

For up sampling, we employ the one-dimensional linear interpolation function to reconcile two time series recorded at 50 Hz and 500 Hz, respectively. Linear interpolation constructs a piecewise-continuous approximation

$$\hat{x}(t) = x_i + \frac{x_{i+1} - x_i}{t_{i+1} - t_i}(t - t_i) \quad (3.3)$$

for  $t \in [t_i, t_{i+1}]$ , ensuring that the interpolant passes exactly through the original samples  $(t_i, x_i)$ . By evaluating  $\hat{x}(t)$  on the finer 500 Hz grid, the lower-rate data are up-sampled without introducing the oscillatory artifacts that higher-order polynomials can induce. This choice of linear scheme offers a balance between computational efficiency and signal fidelity: it preserves first-order continuity, limits overshoot and improves the linearity.

Down-sampling is also used, it reduces a 500 Hz signal to 50 Hz so that the two signals are of equal length and correspond exactly. Uniformly down-sampling lies in the trade-off between data resolution and resource efficiency. Reducing the sampling rate to



---

a coarser sampling rate is sufficient to capture slower dynamics, reduce storage and speed up subsequent computations including feature extraction, classification and data visualization.

---

## 4 Proposed Estimation Methods

---

After the measurement and data pre-processing, the measurement vector  $\mathbf{z}$  is defined as

$$\mathbf{z} = \begin{bmatrix} v_s \\ v_d \end{bmatrix}. \quad (4.1)$$

The force of current sample  $\hat{f}[n_0]$  is estimated based on the  $\mathbf{z}[n]$  using a causal model  $\mathcal{M}$  as

$$\hat{f}[n_0] = \mathcal{M}((\mathbf{z}[n])_{n=1}^{n_0}, \mathbf{w}, \mathbf{b}), \quad (4.2)$$

where  $\mathbf{w}$  and  $\mathbf{b}$  refers to the weights and bias in model  $\mathcal{M}$ .

The objective is to determine the optimal weights  $\mathbf{w}^*$  and bias  $\mathbf{b}^*$  such that the resulting estimate  $\hat{f}[n_0]$  closely approximates the true value  $f[n_0]$  with negligible error

$$f[n_0] \approx \mathcal{M}((\mathbf{z}[n])_{n=1}^{n_0}, \mathbf{w}^*, \mathbf{b}^*). \quad (4.3)$$

After obtaining a sequence of estimates  $\hat{f}$ , we evaluate the errors between the estimate and the actual using the loss function  $\mathcal{L}(f, \hat{f})$ .

L1 loss (MAE) is a common loss function defined as

$$\mathcal{L}_{\ell_1}(f, \hat{f}) = \frac{1}{N} \sum_{i=1}^N |f[i] - \hat{f}[i]|. \quad (4.4)$$

.

MSE Loss is another common function as

$$\mathcal{L}_{\ell_2}(f, \hat{f}) = \frac{1}{N} \sum_{i=1}^N (f[i] - \hat{f}[i])^2. \quad (4.5)$$

---

Stochastic Gradient Descent (SGD) is used to optimize weights for minimum loss

$$\mathbf{w}^*, \mathbf{b}^* = \arg \min_{\mathbf{w}, \mathbf{b}} \mathcal{L}(f, \hat{f}). \quad (4.6)$$

---

## 4.1 Temporal convolutional network

---

Convolutional Neural Networks (CNNs) have garnered significant attention in the realm of one-dimensional (1D) signal processing. They are tailored to handle sequential data and have emerged as a leading approach for several important signal processing tasks, including personalized ECG classification, structural condition monitoring, fault detection in power electronics, and motor diagnostics [24].

1D CNNs can be trained effectively even with limited datasets, reducing the risk of overfitting and making them suitable for scenarios where data acquisition is challenging. Moreover, due to their low computational requirements, 1D CNNs are well-suited for real-time and low-cost applications.

However, classic CNNs lack mechanisms to retain long information of previous inputs. This absence of long memory can be a drawback when modeling temporal dependencies inherent in sequential data. Also, in their typical configuration, CNNs process data using both past and future context, making them non-causal. This characteristic is unsuitable for real-time applications where future data isn't available at the time of processing. For this reason, it's unfair to directly compare the performance of CNN and Kalman models in this experiment.

Bai et al. [25] introduced a causal convolutional neural network architecture that ensures no information flows from future to past, enabling it to map input sequences of any length to output sequences of equal length. Temporal convolutional networks (TCNs) are used in this study. In TCNs, multiple network structures are used. TCN is based on the following two structures

Firstly, the Residual Network proposed by He et al. [26] was applied to solve the problem of decreasing accuracy when the depth of the layers increases in deep learning, effectively eliminating the problem that weights in deeper layers are difficult to train due to degradation or vanishing gradient descent. The idea is to connect the input of a layer directly to the output of a layer after skipping a few connections. Based on the above plain network, shortcut connections are inserted which turn the network into its



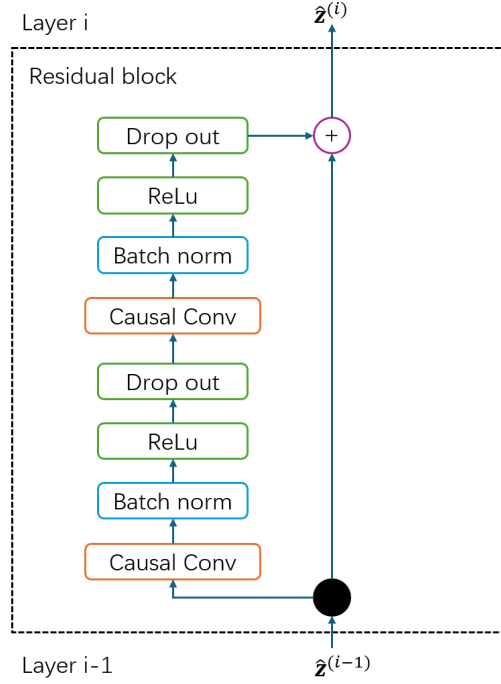


Figure 4.1: TCN Structure

counterpart residual version. The identity shortcuts can be directly used when the input and output are of the same dimensions.

On the original TCN structure, the batch normalization layer is added to each residual block to solve the problem of over 100-fold difference in the order of magnitude of input and output data due to different units. Also, with Batch Normalization, neural networks can be trained using higher learning rates and without strict initialization requirements [27].

The residual block structure of TCN is shown in Fig. 4.1. It is a stack of several layers, each residual block has two convolutional layers, one ReLu activation layer and one batch normalization layer. The number of input channels is  $C_i$ , the number of channels in the hidden layers are  $C_h$ , and the number of channels in the output layer is  $C_o$ .

In TCN, the number of residual blocks is defined as  $L_r$ .

Secondly, following the work of [28], dilated causal convolution layers are implemented in TCN as well. As shown in Fig. 4.2 even the network has many deep layers, the

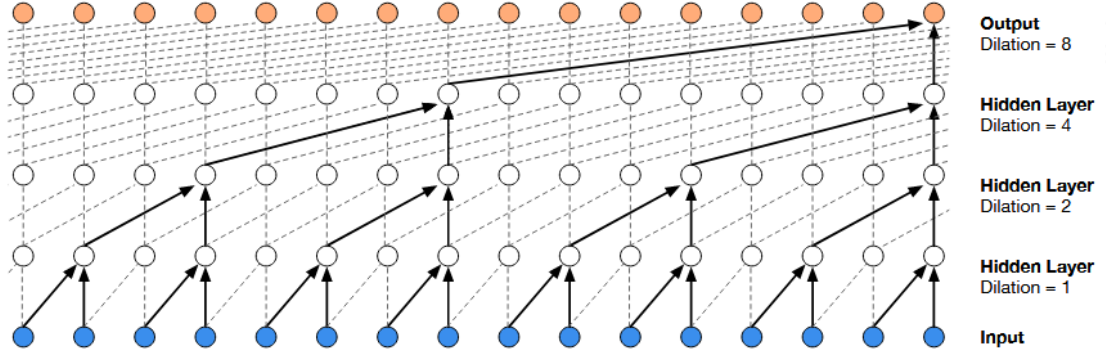


Figure 4.2: A stack of dilated causal convolutional layers. Figure adopted from [28].

network allows an extremely large effective history in deeper layers. The dilated factor  $d$  for layer  $i$  is designed as  $d = O(2^i)$ . This greatly enhances the receptive field of the deeper layers as  $(S_k - 1)d$ , where  $S_k$  is the convolution kernel size.

#### 4.1.1 Datasets

To construct a dataset, the data that is previously denoised by low-pass filtering, then synchronised. Since the TCN is a data-driven model, a larger data volume can improve the training performance. Therefore, the reference force data is interpolated to reach the same size of  $v_s$  and  $v_d$ . Afterwards, it is sliced into data segments of  $S = 20,000$  samples in length. Thereby we obtain a dataset  $\mathcal{D}_{\text{train}}$  of size  $|\mathcal{D}_{\text{train}}| = 89$  as a training set. And a dataset  $\mathcal{D}_{\text{validation}}$  of size  $|\mathcal{D}_{\text{validation}}| = 3$  as a validation set. We also set up two unsliced validation sequence of length  $N_s = 70k$  and  $N_l = 700k$ . Part of the training set and the validation set can be seen in the Fig. 4.3 and Fig. 4.4.

#### 4.1.2 Training and validation

The measurement vector  $\mathbf{z}$  in Eq. (4.1) is used as the input of TCN.

After forward propagation, the output is the estimated force  $\hat{f}$ .

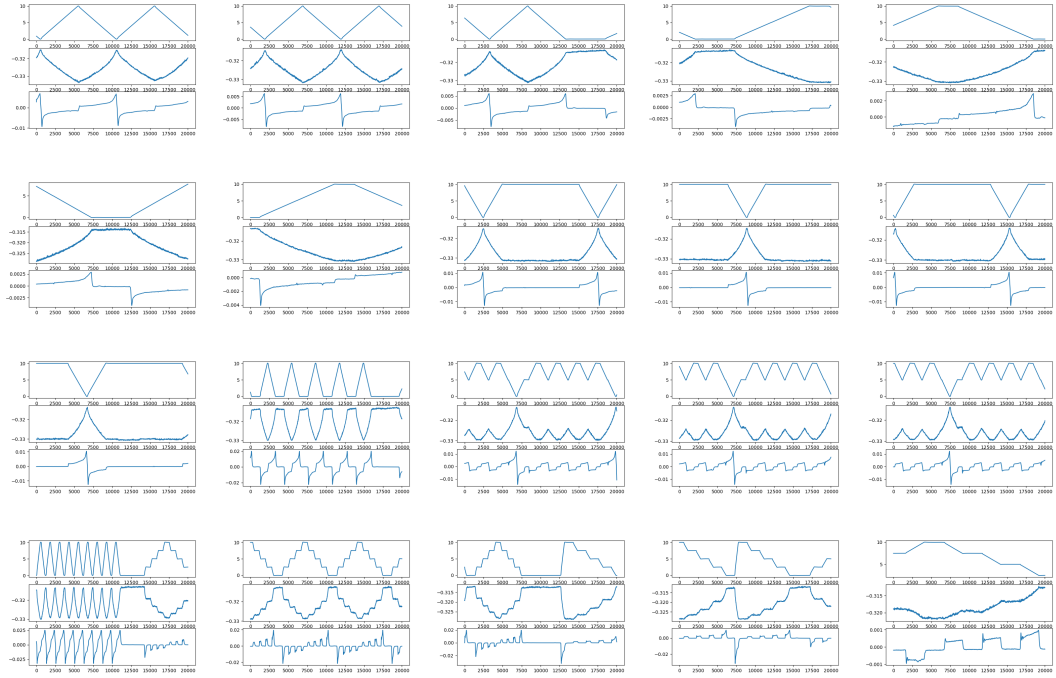


Figure 4.3: Part of the training Sets

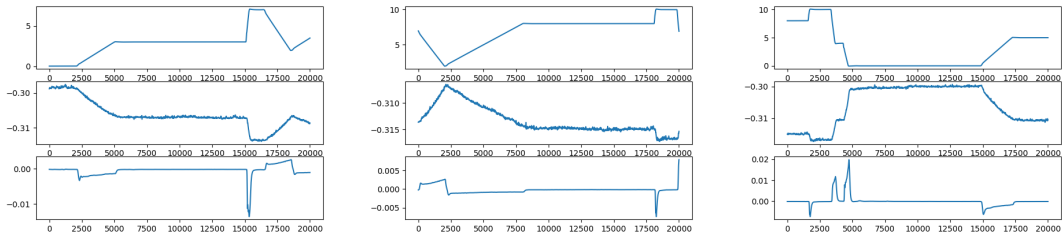


Figure 4.4: Part of the validation Sets

---

Both L1 loss and MSE loss are used in the experiment, it was found that MSE Loss performed better in the training set, but exhibited apparent estimation variance in the validation set, appearing especially in the part that is supposed to be static.

One possible explanation is that MSE loss gives more weight to large deviations compared to small errors, whereas L1 loss treats them equally. This means that for input data, the L1 loss is more likely to ignore outliers. It is inevitable that measurement variability will be mixed in our data due to external mechanical vibrations (e.g., from the floor or table). The L1 loss better circumvents the prediction noise due to these outliers.

In the training step, the loss is then propagated backward through the network, adjusting the weights and biases of each layer.

---

## 4.2 Deep learned Kalman Filter

---

Kalman filter (also known as linear quadratic estimation) is an optimal recursive estimator based on the Minimum Mean Square Error (MMSE). It models the system using a linear state-space representation, where the state estimate is updated in two steps: prediction and update.

In the prediction step, the prior estimate is propagated forward using the system dynamics. The current state  $x[n]$  is predicted based on the previous state  $x[n - 1]$  as

$$\mathbf{x}[n] = \mathbf{A}[n]\mathbf{x}[n - 1] + \mathbf{w}[n] + \mathbf{B}[n]\mathbf{u}[n], \quad (4.7)$$

where  $\mathbf{A}$  is the prediction matrix,  $\mathbf{B}$  is the control-input matrix,  $\mathbf{u}$  is the control vector and  $\mathbf{w}$  is process noise with covariance  $\mathbf{Q}$ . Covariance matrix  $\mathbf{P}[n]$  is predicted as

$$\mathbf{P}[n] = \mathbf{A}[n]\mathbf{P}[n - 1]\mathbf{A}[n]^T + \mathbf{Q}[n], \quad (4.8)$$

where  $\mathbf{H}$  is the measurement to prediction matrix. In the update step, the estimate is refined using measurements, with the Kalman gain determining the optimal weighting to minimize the posterior error covariance. The predicted measurement vector  $\mu$  is updated as

$$\mu[n] = \mathbf{H}[n]\mathbf{x}[n]. \quad (4.9)$$

Covariance of the predicted measurement is updated as

$$\Sigma = \mathbf{H}[n]\mathbf{P}[n]\mathbf{H}[n]^T. \quad (4.10)$$

The optimal Kalman gain at point  $n$  is

$$\mathbf{K}[n] = \mathbf{P}[n]\mathbf{H}[n]^T(\mathbf{H}[n]\mathbf{P}[n]\mathbf{H}[n]^T + \mathbf{R}[n])^{-1}. \quad (4.11)$$

After that, the Kalman gain is used to update the state estimation and covariance estimation as

$$\mathbf{x}[n]' = \mathbf{x}[n] + \mathbf{K}[n](\mathbf{z}[n] - \boldsymbol{\mu}[n]), \quad (4.12)$$

$$\mathbf{P}[n]' = (\mathbf{I} - \mathbf{K}[n]\mathbf{H}[n])\mathbf{P}[n]. \quad (4.13)$$

Assuming Gaussian noise, the filter efficiently fuses prior estimates and noisy observations to produce the best linear unbiased estimate of the system state.

### 4.2.1 Model of Kalman Filter

We propose the following state model for the system.

The state vector  $\mathbf{x}$  consists of 5 variables, the force  $f$ , change in length of material after being pressed  $\Delta l$  of sample  $n$ ,  $n - 1$  and  $n - 2$  and the variable  $h$  indicates the continuous effect of the previous force on the sensor

$$\mathbf{x}[n] = [f[n], \Delta l[n], \Delta l[n - 1], \Delta l[n - 2], h[n]]^T. \quad (4.14)$$

In the prediction phase, the force is

$$f[n] = f[n - 1] + \Delta f. \quad (4.15)$$

Assuming that the change in force is related to the current deformation, the first order and the second order derivative of the deformation exists as follows

$$f[n] = k_1[n]\Delta l[n] + k_2[n](\Delta l[n - 1] - \Delta l[n - 2]) + k_3[n](\Delta l[n] - 2\Delta l[n - 1] + \Delta l[n - 2]). \quad (4.16)$$

$k_1, k_2, k_3$  represent the coefficients of the length change of the material which effects the change in electrical resistance, and they should be nonlinear.

Therefore, the change in length is

$$\Delta l[n] = \left( \frac{1}{k_1[n] + k_2[n] + k_3[n]} \right) [f[n] + (k_2 + 2k_3)\Delta l[n-1] + k_3\Delta l[n-2]]. \quad (4.17)$$

Meanwhile, the previous length change is update as

$$\Delta l[n-1] = \Delta l[n], \quad (4.18)$$

$$\Delta l[n-2] = \Delta l[n-1]. \quad (4.19)$$

Assuming that the change in  $h$  is non-linear, a separate neural network parameter  $\theta_h$  is used to predict it as

$$h[n] = h[n-1] + \theta_h[n]. \quad (4.20)$$

Therefore, the prediction matrix  $\mathbf{A}$  is

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ \frac{1}{k+k_2+k_3} & \frac{k_2+2k_3}{k+k_2+k_3} & \frac{k_3}{k+k_2+k_3} & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad (4.21)$$

State update equation is Eq. (4.7) with control vector  $\mathbf{u}[n] = [0, 0, 0, 0, \theta_h[n]]^T$  and control-input matrix

$$\mathbf{B}[n] = \begin{bmatrix} \mathbf{0}_{4 \times 5} \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

The process noise is assumed to be drawn from a zero mean normal distribution as

$$\mathbf{w} = \Delta f \begin{bmatrix} 1 \\ \frac{1}{k_1+k_2+k_3} \\ 0 \\ 0 \\ 0 \end{bmatrix}, \Delta f \sim \mathcal{N}(0, \sigma_f^2). \quad (4.22)$$

Then the covariance of  $\mathbf{w}$  is

$$\mathbf{Q} = \mathbf{E}(\mathbf{w}^T \mathbf{w}) \quad (4.23)$$

Therefore, the covariance matrix  $\mathbf{Q}$  of the process noise is

$$\mathbf{Q} = \begin{bmatrix} 1 & k_i & 0 & 0 & 0 \\ k_i & k_i^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \sigma_f^2, k_i = \frac{1}{k_1 + k_2 + k_3}. \quad (4.24)$$

In the update phase, the observation is used to adjust the prediction.

According to Eq. (2.2) and

$$f_d[n] = k_2[n](\Delta l[n] - \Delta l[n-1]) + k_3[n](\Delta l[n] - 2\Delta l[n-1] + \Delta l[n-2]), \quad (4.25)$$

the measurement  $v_d$  is denoted as

$$v_d[n] = (1 + \frac{R_2}{R_1}) \frac{d_{33}}{c_m} [(k_2[n] + k_3[n])\Delta l[n] + (-k_2[n] - 2k_3[n])\Delta l[n-1] + k_3[n]\Delta l[n-2]] \quad (4.26)$$

Similarly, according to Eq. (2.5) and

$$f_s[n] = k_1[n]\Delta l[n], \quad (4.27)$$

the measurement  $v_s$  is denoted as

$$v_s[n] = \frac{\rho v_0}{4RA} \Delta l[n] \quad (4.28)$$

Therefore, measurement to prediction matrix  $\mathbf{H}$  is

$$\mathbf{H} = \begin{bmatrix} 0 & \theta_1 & 0 & 0 & 0 \\ 0 & \theta_2(k_2 + k_3) & -\theta_2 k_2 - 2\theta_2 k_3 & \theta_2 k_3 & 0 \end{bmatrix}, \quad (4.29)$$

where  $\theta_1$  and  $\theta_2$  are coefficients denoted as

$$\theta_1 = \frac{\rho v_0}{4RA}, \theta_2 = (1 + \frac{R_2}{R_1}) \frac{d_{33}}{c_m}. \quad (4.30)$$

Covariance matrix of the uncertainty of the measurement  $\mathbf{R}[n]$  mainly comes from the

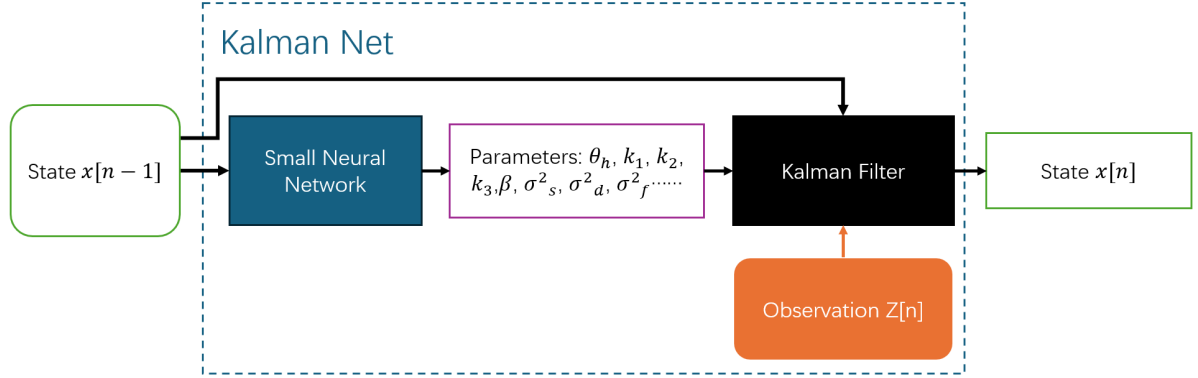


Figure 4.5: Structure of Kalman Net

noise of sensor and the interference of experiment environment. It is denoted as

$$\mathbf{R}_n = \begin{bmatrix} \sigma_s^2 & 0 \\ 0 & \sigma_d^2 \end{bmatrix}, \quad (4.31)$$

where  $\sigma_s^2$  refers to the variance of the measurement data  $v_s$  and  $\sigma_d^2$  refers to the variance of measurement data  $v_d$ .

## 4.2.2 Kalman Net

As shown in the Fig. 4.5, is the structure of the deep learning Kalman filter.

The Kalman filter is a part of neural network. In one iteration, the past state  $\mathbf{x}[n]$  and current observation  $\mathbf{z}[n]$  are put into a small neural network, after hidden and activation layers, the output values are used as parameters for Kalman filtering, then  $\mathbf{x}[n-1]$  and  $\mathbf{z}[n]$  are put into this Kalman filter to perform the prediction and updating steps then get the latest state  $\mathbf{x}[n]$ . The forward propagation process can be defined as following. The hidden layer is

$$\mathbf{h} = \mathbf{R}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad (4.32)$$

where  $\mathbf{R}(\cdot)$  denotes the element-wise ReLu activation function, defined by

$$\mathbf{R}(z) = \max(0, z). \quad (4.33)$$



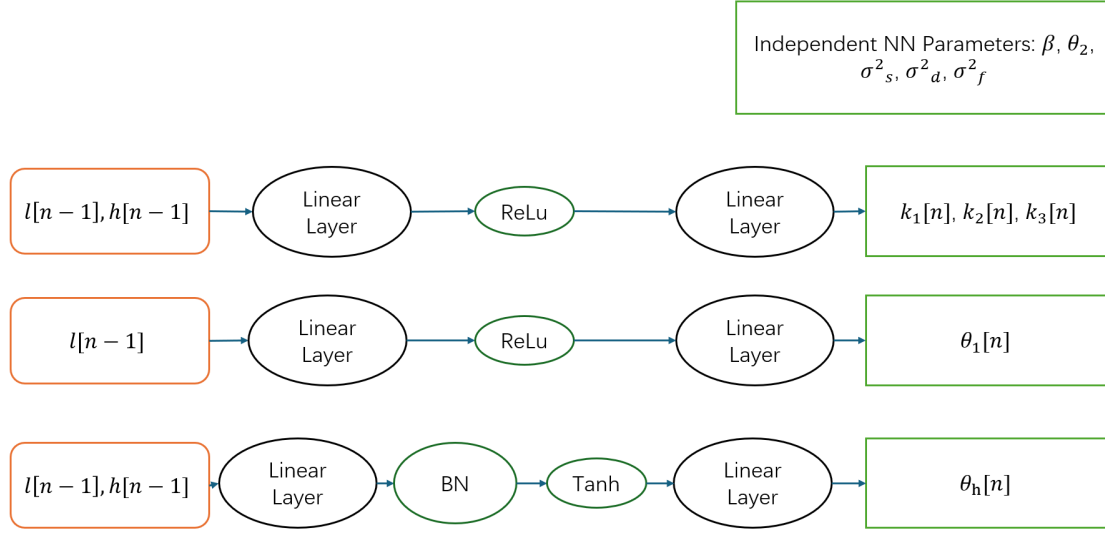


Figure 4.6: Layers structure

Output layer is

$$y = \mathbf{W}_2 \mathbf{x} + \mathbf{b}_2. \quad (4.34)$$

The overall computation performed by the small network can thus be written compactly as

$$\varphi(\mathbf{x}) = \mathbf{W}_2 R(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2. \quad (4.35)$$

### 4.2.3 Neural network layer structure

Table 4.1: Initial values of network parameters

| Parameter                | Input            | Hidden layer size | Out put layer size |
|--------------------------|------------------|-------------------|--------------------|
| $k_1[n], k_2[n], k_3[n]$ | $l[n-1], h[n-1]$ | (2,64)            | (64,1)             |
| $\theta_1[n]$            | $l[n-1]$         | (1,64)            | (64,1)             |
| $\theta_h[n]$            | $l[n-1], h[n-1]$ | (2,32)            | (32,1)             |

As shown in the Tab. 4.1 and Fig. 4.6,  $k_1[n], k_2[n], k_3[n], \theta_1[n]$  and  $\theta_h[n]$  are the parameters that were trained by small neural network based on the current state. In addition, there are  $\beta, \theta_2, \sigma_s, \sigma_d$  and  $\sigma_f$ , which are independent and always invariant within one network.

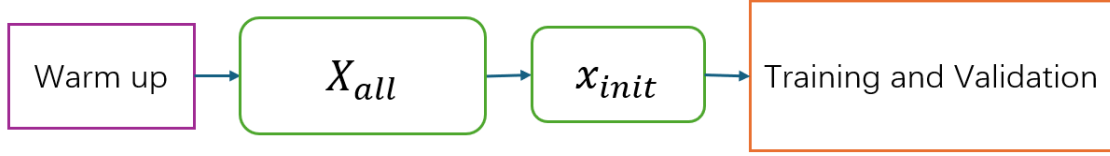


Figure 4.7: Flowchart of the warm-up process

$\beta$  refers to the bias factor for the observed value of  $v_s$  such that it takes 0 in the initial unforced state.

In particular, for the parameter  $\theta_h$ , a Batch Normalization layer was added after the hidden layer to replace the ReLu activation function in order to allow its output to be negative and to introduce more nonlinearity.

In the following discussion, we refer to the model as DLKF-FFS.

#### 4.2.4 Training and validation

Since Kalman filtering is iterative, the training set and the validation set theoretically can be of any length. Directly training the neural network on the entire sequence can be computationally expensive and more prone to gradient vanishing and gradient explosion. To address this, the signal is partitioned into smaller non-overlapping slices, each treated as an individual training sample, exactly as we do to get the training set for TCNs. However, we chose to down sample due to the fact that the training takes too long in the for loop. During the training process, each batch is of shape (5,2,2000), i.e., batch size is 5,  $v_s$  and  $v_d$  as 2 inputs and the signal length is 2000.

##### Warm-up process

The warm-up process is designed to combat the lack of temporal information associated with training on sliced data.

The observed data i.e.  $v_s$  and  $v_d$  are time domain signals. Therefore, the observations are slightly different when time is different, even when exactly the same force  $F$  is applied.

---

For the Kalman filter, the missing time information can be compensated by inputting initial states. The initial state includes  $l_n$  and  $h_n$ , which means that the deformation and hysteresis information that the current sensor is subjected to is known. This effectively eliminates inconsistencies in the force sensitivity of the sensor due to the effects of prolonged hysteresis.

### Training using Truncated Backpropagation

The warm-up process is applied only when the number of training steps is greater than  $N_{\text{warmup}} = 10$ , the whole training process is as follows:

1. Using the current model without gradient, input the observed signals  $v_s$  and  $v_d$  from beginning to end.
2. After iteration, the states of all points are predicted from beginning to end and stored in  $\mathbf{X}_{all}$ .
3. Start forward propagation with input training set of batch size 5 segments of size (2,2000), for each segment, find the state  $\mathbf{x}_{init}$  of its initial point from  $\mathbf{X}_{all}$ .
4. The  $l_n$  and  $h_n$  obtained from  $\mathbf{x}_{init}$ , and  $v_s$ ,  $v_d$  from the training set are applied to the neural network mentioned before, and the training is done to obtain Kalman filter with several parameters.
5. These parameters along with  $\mathbf{x}_{init}$  are fed into the Kalman filter and iterated to obtain the estimation  $\hat{f}$ .
6. The loss of this data fragment is calculated. Then back propagation.

Fig. 4.7 shows the warm-up process.

### Initialization

During the training, it was found that the initialization of parameters in neural networks and the weights and biases of each layer significantly influence the efficiency and stability of the training process. Proper initialization helps prevent issues of vanishing or exploding gradients, which can hinder or entirely stall the learning process. With reasonable initialization, networks can converge faster and achieve better generalization.

Since the parameters of DLKF-FFS are interpretable, as opposed to convolutional neural networks, we can easily estimate these values based on the magnitude of the coefficients in the physical model, so setting the initial values of the parameters and weights in the network is necessary and convenient.

In order to obtain reasonable initial values  $\hat{k} = k[n] = \text{const.}$ ,  $\hat{\theta}_1 = \theta_1[n] = \text{const.}$  and other parameters of the Kalman filter are looked for in the classic Kalman filter. The Nelder-Mead method was used to search the optimal parameter values corresponding to the lowest loss achieved by the classic Kalman filter where the parameters are kept constant.

The initial value is shown in the Tab. 4.2 and Tab. 4.3.

Table 4.2: Initial values of network parameters

| Parameter    | Initial Value |
|--------------|---------------|
| $\beta$      | 0.40          |
| $\theta_2$   | 1.05          |
| $\sigma_s^2$ | 2.5           |
| $\sigma_d^2$ | 0.5           |
| $\sigma_f^2$ | 1.5           |

Table 4.3: Initial values of network parameters

| Layer                   | Initial Weight | Bias |
|-------------------------|----------------|------|
| Output layer $k_1$      | -0.24          | 0    |
| Output layer $k_2$      | 0.02           | 0    |
| Output layer $k_3$      | -0.02          | 0    |
| Output layer $\theta_1$ | 0.03           | 0    |
| Output layer $\theta_h$ | 0.01           | 0    |

### Offset correction and penalty loss function

In the base model, the parameter  $\theta_h$  is only inside the neural network like the other parameters, and  $h_n$  is a value inside the state vector. In the following method,  $h_n$  is directly used to correct the offset of the estimates outside the Kalman filter.

---

Assuming that the sustained effect of previously applied force can be approximated as a residual deformation  $\Delta l_0$ , the true force can be expressed as

$$f[n] = k_1(\Delta l[n] - \Delta l_0) + k_2(\Delta l[n-1] - \Delta l[n-2]) + k_3(\Delta l[n] - 2\Delta l[n-1] + \Delta l[n-2]). \quad (4.36)$$

In contrast, the base model estimates the force according to Eq. (4.19):

$$\hat{f}[n] = k_1 \Delta l[n] + \dots \quad (4.37)$$

where the residual deformation  $\Delta l_0$  is not accounted for, leading to a constant offset in the force estimation.

To directly compensate for this offset and improve estimation accuracy, the model output can be corrected as follows:

$$\hat{f}'[n] = \hat{f}[n] - k_1 \Delta l_0. \quad (4.38)$$

If  $\Delta l_0$  is approximated by  $h[n-1]$ , the final corrected estimate becomes:

$$\hat{f}'[n] = \hat{f}[n] - k_1 h[n-1]. \quad (4.39)$$

The penalty function is used to penalize cases where the parameter  $\theta_h$  takes a positive value when the state variable  $l_n$  is small. The purpose is to better allow the model to find the right direction in training. The parameters  $\lambda$  and  $\epsilon$  are key to the penalty loss  $\mathcal{P}$ . They determine how much and when the penalty loss should be added.

$$\mathcal{P} = \lambda \frac{1}{i} \sum_i \mathbb{1}(|l_i| < \epsilon) \text{ReLU}(\theta_h^{(i)}) \quad (4.40)$$

The penalty function can be incorporated into both the base model and the offset-correction variant during training. Specifically, the total loss is computed as the sum of the primary loss and the penalty term, which jointly guide the model updates via back-propagation. However, during validation, the penalty function is excluded from the loss computation to provide an unbiased assessment of model performance.

---

## 4.3 Real time implementation

---

To show the results of TCN and DLKF-FFS's signal processing, the proposed algorithms are implemented for real-time use.

In the hypothetical scenario, the user applies arbitrary pressure to the sensor while the system continuously displays the real-time voltage waveforms  $v_s$  and  $v_d$ . Concurrently, we plan to compute and present the corresponding force estimates produced by the model. By employing a pretrained TCN model or DLKF-FFS, the module processes incoming data at a rate of 500 samples per second and incurs virtually zero latency, thereby enabling real-time force estimation and filtering.

Specifically, for the TCN model, the implementation steps are as follows:

1. A low-pass filter is applied to pre-process the input data.
2. Warm up: At the beginning, in order to ensure good results, it is necessary to wait for enough input samples, e.g. 40s or 20k samples.
3. Every time after a refresh period  $\tau$ , read the last 20k sample points to do the prediction by a pre-trained model(it takes about 0.05s), find the start and end time point of the new input, and make the corresponding output.

In this step, only the last part of the predicted 20k points is adopted, for example, the sampling rate of the data collection is 500Hz and the refresh period  $\tau$  of the method is set to 0.2s, then except for the first time in the warmup process, only the last 100 points are adopted after each prediction, and the overlap reaches 95%.

Such long segments of overlap can be achieved due to the advantage of the model's prediction speed being faster than the sampling speed. The long overlap ensures the consistency of the output data and eliminates the inaccuracies caused by the small amount of data at the beginning of the causal model.

Meanwhile, the application process for DLKF-FFS is as follows:

1. Data preprocessing of the input signal as above.
2. No warm-up is required because DLKF-FFS always has consistent output for inputs of arbitrary length.

- 
3. Every time a refresh cycle  $\tau$  passes, the input signal and state  $\mathbf{x}_{n-1}$  are provided to the pre-trained model, and the result is predicted which takes about  $0.038\tau$  seconds. Output the results and record the ending state  $\mathbf{x}_n$ .

Since it is based on the Kalman filter, the process of state iteration results in a consistent output for any input signal length. Thanks to the contribution of state retention, DLKF-FFS can maintain consistency without overlap. This results in a shorter computational delay compared to TCN and the advantage of not needing to warm up for a long time.

---

## 5 Experimental Results

---

---

### 5.1 Temporal Convolutional Neural Network (TCN)

---

Table 5.1: Performance of TCN(L1 loss)

|                     | Training loss | Short validation loss | Long validation loss |
|---------------------|---------------|-----------------------|----------------------|
| TCN-8               | 0.15          | 0.24                  | 0.75                 |
| TCN-8 without $v_d$ | 0.22          | 0.30                  | 1.11                 |
| TCN-12              | 0.14          | 0.34                  | 0.74                 |

We evaluate several TCN configurations. Configuration TCN-8 consists of  $L_r = 8$  layers with  $C_h = 32$  channels for each 1D convolution. TCN-12 consists of  $L_r = 12$  layers with  $C_h = 32$  channels. Since there are two convolutional layers in every residual block and the kernel size  $S_k = 3$ , the receptive field is calculated as follows:

$$\text{RF} = 1 + \sum_{i=0}^{L_r-1} 2(S_k - 1)2^i. \quad (5.1)$$

Therefore, the receptive field of TCN-8 is 1021 and TCN-12 is 16381. The result of TCN based on L1 loss training is presented in Tab. 5.1 and Tab. 5.2. Here the lengths of the

|                                       | HER    | DER    |
|---------------------------------------|--------|--------|
| Maximum                               | 12.38% | 0.73‰  |
| Average                               | 8.04%  | 0.62‰  |
| Improvement compared to the reference | 43.58% | 29.22% |

Table 5.2: Hysteresis and drift error of the estimates from TCN-8



---

---

|          | Training Loss | Short Val Loss | Long Val Loss |
|----------|---------------|----------------|---------------|
| L1 loss  | 0.15          | 0.24           | 0.75          |
| MSE loss | 0.03          | 0.24           | 0.77          |

Table 5.3: Comparison of L1 loss and MSE loss on TCN-8

long and short validation sequences are respectively 70k and 700k which is mentioned in Sec. 4.1.1. It can be seen that more layers do not result in increased performance, especially on the short validation sequence, while for the long validation sequence both are similar. TCN-8 without  $v_d$  input performs worse than the one with  $v_d$ . It converges at a higher loss on training data set and has a bad generalization performance on validation set. It can be concluded that the inclusion of  $v_d$  as an input variable is beneficial in reducing the estimation error.

As shown in Tab. 5.3, we trained and validated the model using both L1 loss and MSE loss. While the performance on the validation sequences is numerically similar for both loss functions, models trained with MSE loss exhibit faster convergence. This partially explains the widespread adoption of MSE in similar estimation tasks.

However, it is difficult to determine definitively which loss function is superior. Notably, the MSE loss on the training dataset is significantly lower, which may indicate a higher risk of overfitting. Based on this observation, we focus our subsequent analysis on models trained with L1 loss, due to its potentially better generalization behavior.

As Fig. 5.1 shown, in validation data of length 70k samples, i.e., 140 seconds, the estimates are generally accurate, with good retention of turning points in the data. However, a lot of high-frequency noise exists in the first 100-200 points at the beginning. This is mainly due to the fact that as a temporal causal model, only  $n$  signals are accessible in the prediction of the first  $n$  points, and the receptive field is mostly empty. Another conclusion is that it also generalizes well, especially for values such as 8N, 3N that basically do not appear in the training set.

However, when faced with long input signals of length over 700k, i.e. 1400s, as shown in the Fig. 5.2, TCN model does not perform as well as when faced with shorter ones. In long signals, L1 loss is 3.16 times higher compared to the short validation sequence..

One explanation for this situation is that the length of the training set it is exposed to is 20k, so it performs better with the same shorter signal. For long signals, on the other hand, it does not have a large enough receptive field so it is unable to extract some of the features in signals that are long such as the changes in  $d_{33}$  under long-term forces.

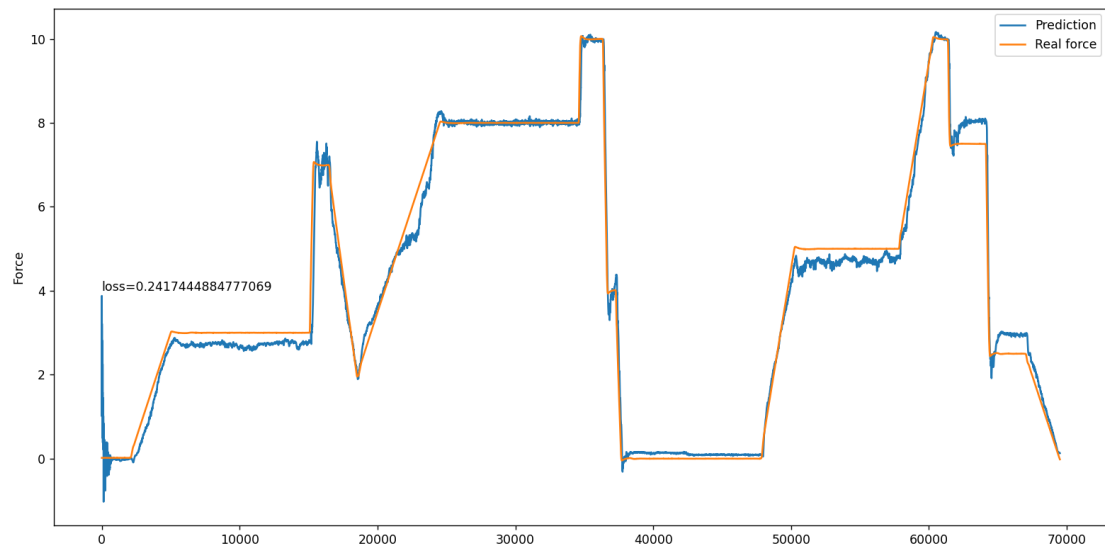


Figure 5.1: 8-Modules TCN model prediction on short validation data

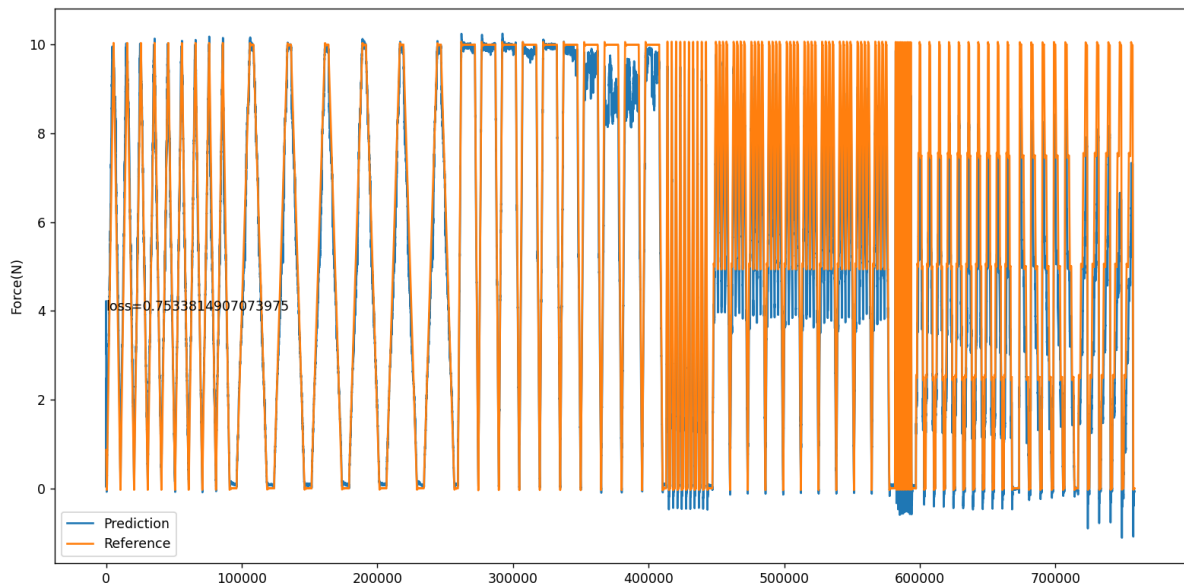


Figure 5.2: 8-Modules TCN model prediction on a long validation data

Also, as time increases, and later measurements vary more due to the long-term changes in the sensor characteristic and long-range dependencies, showing inconsistency from the beginning.

In the real application scenario, force-sensor readings arrive as continuous, variable-length voltage streams rather than fixed-length clips. The choice of “short” versus “long” validation sequences merely probes the model’s sensitivity to different temporal scales; it does not dictate a rigid requirement on input length in practice.

Considering the level of hysteresis and drift, compared to the reference data, the TCN-8 reduces about 43% of the hysteresis error and 29% of the drift error per second. Although TCN is a black-box model, it can be concluded that the reason it eliminates both errors is that it was able to extract features, especially the variation of  $v_d$  in relation to the force. There is a complex mathematical relationship between them, TCN simulates this non-linear relationship through a large number of hidden layer weights, which can reduce the error in prediction. On the other hand, due to the simultaneous action of drift and hysteresis, it is very difficult to describe the relationship between input and output with a physical model. The TCN, as a totally model-agnostic method, can learn to reduce both these effects given a large enough receptive field.

## 5.2 DLKF-FFS

Table 5.4: Result of DLKF-FFS with different methods(L1 loss)

| Method                 | Penalty | Training Loss | Short Val Loss | Long Val Loss |
|------------------------|---------|---------------|----------------|---------------|
| Base model             | Yes     | 0.36          | 0.38           | 0.59          |
|                        | No      | 0.39          | 0.48           | 0.60          |
| Base model (no $v_d$ ) | Yes     | 0.63          | 0.53           | 0.66          |
| Offset correction      | Yes     | 0.45          | 0.39           | 0.45          |
|                        | No      | 0.57          | 0.46           | 0.60          |

As can be found in Tab. 5.4 and Tab. 5.5, different methods are used when training DLKF-FFS.

It can be noticed that the model with direct offset correction has a lower loss during training, but can not perform as well as the base model on the validation set. In addition, the penalty function not only makes the convergence of the training process faster, but

|          | Training Loss | Short Val Loss | Long Val Loss |
|----------|---------------|----------------|---------------|
| L1 loss  | 0.36          | 0.38           | 0.59          |
| MSE loss | 0.25          | 0.37           | 0.80          |

Table 5.5: Comparison of L1 loss and MSE loss on DLKF-FFS

|                                       | HER    | DER(per second) |
|---------------------------------------|--------|-----------------|
| Maximum                               | 10.71% | 1.11‰           |
| Average                               | 7.90%  | 0.72‰           |
| Improvement compared to the reference | 44.62% | 17.9%           |

Table 5.6: Hysteresis and drift error of the estimates from DLKF-FFS

also avoids the error situation where the value of the parameter  $h$  is always positive or always negative, so it also performs better on long validation sets.

The model variant excluding the  $v_d$  input exhibits substantially higher loss on both the training and validation datasets compared to the original model. This highlights the critical role of  $v_d$  as an observation in the Kalman filter, which significantly contributes to reducing the estimation error.

Fig. 5.3 illustrates the estimation results of the base model on a short validation sequence, trained with the penalty function. Both L1 loss and MSE loss are employed as training objectives. Upon comparison, the numerical results are similar, with L1 loss equal to 0.38 and MSE equal to 0.37. The overshoot is more pronounced in the MSE loss-based outcome model.

In Fig. 5.4, the model based on L1 loss with direct offset correction and penalty function performs well on a long validation sequence, especially on the later sample points. Since the value of  $h$  increases slowly with the application of force in the experiment, it becomes larger in the later stages. Therefore, the prediction error can be compensated by directly intervening in the offset correction. However, it is also clear that it is overestimated overall, especially for processes that change particularly rapidly.

As shown in Tab. 5.6, in terms of hysteresis elimination, DLKF-FFS presents good results and it can be found that the maximum and the average of the eliminated hysteresis are similar. However, in terms of drift elimination, higher noise is present in the sequences for the calculation. Therefore, a loss pass filter is used on the estimates before we do the drift calculation. After calculation, it was found that the difference between the maximum and minimum DER is huge. Another phenomenon that exists is that

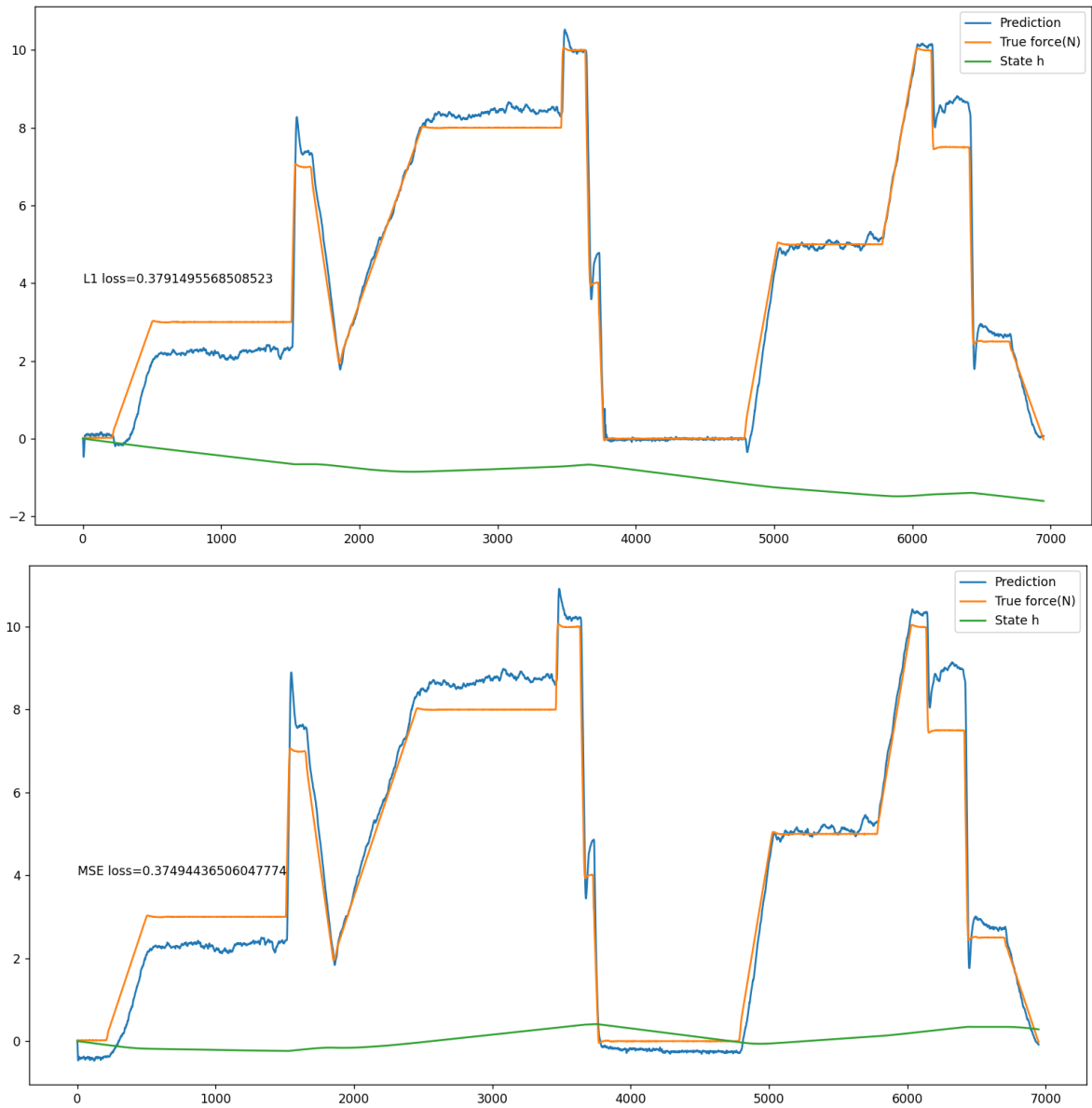


Figure 5.3: DLKF-FFS prediction on short validation sequence based on L1 loss (top) and MSE loss (bottom)

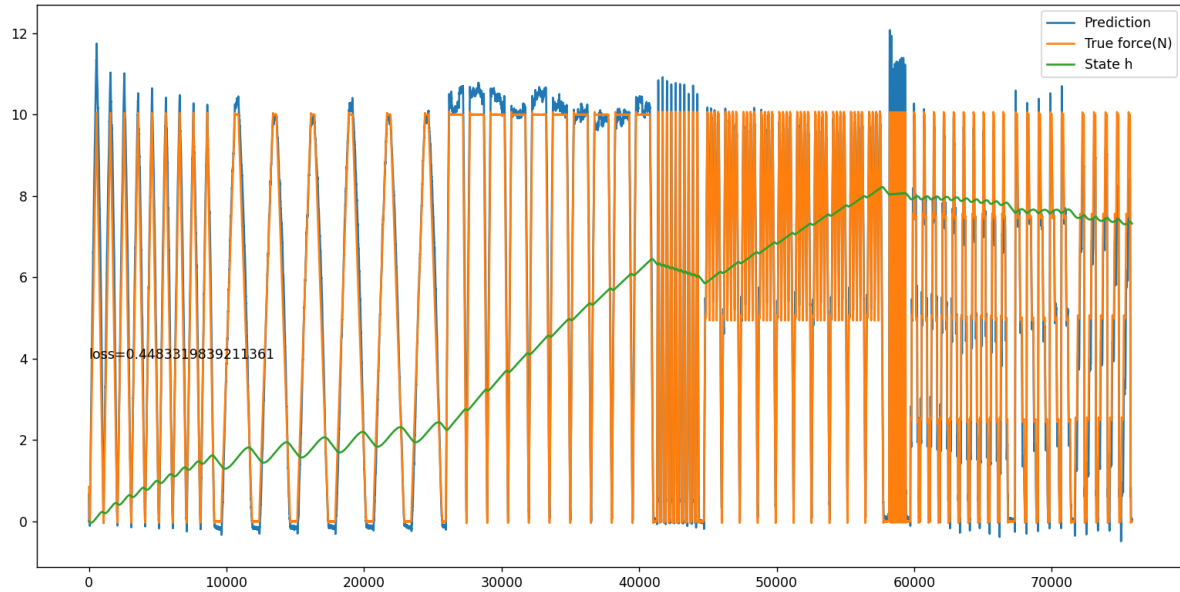


Figure 5.4: DLKF-FFS prediction on long validation sequence based on L1 loss

overshoots appear in the estimates of DLKF-FFS, when there is a sudden change in force. This also makes it very difficult to determine the drift level. Overall, the drift level is reduced compared to the unprocessed reference data, however DLKF-FFS tends to underestimate the applied force during periods of force application.

### 5.3 Comparison

After analyzing their results separately, as shown in the Tab. 5.7, we can compare and analyze their strengths and weaknesses.

DLKF-FFS has 1% the number of parameters of TCN-8. Moreover, these parameters are more interpretable than TCNs, where the weights correspond to parameters that are explicit and the physical meanings of the parameters are understandable. After finding the suitable initialization parameters and weights, the convergence speed is about the same for DLKF-FFS and TCN, both can be close to convergence after about 10 epochs with batch size of 5.

Table 5.7: Comparison between TCN and DLKF-FFS

| Aspect                          | TCN                     | DLKF-FFS                |
|---------------------------------|-------------------------|-------------------------|
| Short sequence performance      | Better                  | Worse                   |
| Long sequence performance       | Worse                   | Better                  |
| Hysteresis elimination          | Worse                   | Better                  |
| Drift elimination               | Better                  | Worse                   |
| Number of parameters            | 98.8k                   | 1,099                   |
| Prediction speed                | Slower                  | Faster                  |
| Interpretability                | Black box               | Fully interpretable     |
| Real-time implementation        | Feasible                | Feasible                |
| Use of physical model knowledge | Model-agnostic          | Required                |
| Noise tolerance                 | Nonlinear, non-Gaussian | Nonlinear, Gaussian     |
| Data requirements               | Fully data-driven       | Low requirements        |
| Training process                | Easier                  | Requires initialization |

For both methods, the loss-metric is higher for the validation data compared to the training data. And both perform worse when dealing with long datasets. This may be due to the fact that the data in the long dataset is not sufficiently consistent overtime, as the data might be time-dependent.

Although TCN achieves lower loss on short sequences, DLKF-FFS demonstrates superior performance on long sequences, achieving a 39.19% lower loss.

Both methods can remove hysteresis and drift, TCN does a good job of removing both, while DLKF-FFS is better at removing hysteresis, and its drift elimination is more difficult to assess due to the noise and overshoots that often occur. They behave very differently when eliminating drift. DLKF-FFS tending to underestimate the prediction of the force as soon as it is applied. TCN, on the other hand, produces a constant estimate or even decreases the estimates during the phase of stable force application.

Although the two models operate based on different mechanisms, they both rely on the inclusion of the  $v_d$  input to improve estimation accuracy. The variable  $v_d$  effectively captures variations in the applied force, providing useful information about its rate and magnitude of change. In the TCN-based model,  $v_d$  is incorporated as part of the input sequence for feature extraction and direct force estimation. In contrast, DLKF-FFS treats  $v_d$  as an observation within the Kalman filter framework, where it is used to update the state via the Kalman gain.

---

The TCN model performs better when confronted with short data, partly because it has more parameters and the neural network is much deeper, which allows it to discover more complex to make correct predictions. It retains the strong robustness of the convolutional neural network while the dropout function ensures that it does not over-fit along with the structure of the residual block which guarantees good generalization performance. At the same time, the large enough receptive field obtained by the dilated function makes it possible to detect features such as noise, hysteresis, drift, etc. and eliminate them efficiently.

Although TCN has more parameters, its prediction speed is faster. For a long sequence with a time length of 1400 seconds, it takes 1.7s to complete the prediction of the sequence, which means that there is only 1.2ms delay in predicting 1 second of data. In contrast, DLKF-FFS takes 54 times longer than it does, taking 93s to predict a sequence of the same length, which means a delay of 38ms per second of data. The above conclusions are based on the results of an average device running a Python implementation with the parameters shown in the Tab. 5.8.

Table 5.8: Computer Configuration

|             |                         |
|-------------|-------------------------|
| Environment | Python 3.9              |
| CPU         | AMD Ryzen 5 5600H       |
| GPU         | NVIDIA GeForce RTX 3050 |

This is mainly due to the fact that in DLKF-FFS, a large, not parallelizable for-loop is used to iterate and loops are generally very inefficient in Python (as compared to compiled languages). Meanwhile, for TCN, the input data is treated as a whole and multiple simple matrix operations are performed, which is fast for GPU.

Both approaches can be used for real-time prediction, thanks to the good generalization performance, and relatively fast prediction speed. With a suitable pre-trained model, a real-time input-prediction-output process can be achieved.

In terms of the ability to handle noise, DLKF-FFS is similar to standard Kalman filter which assumes that the observation noise is Gaussian white noise. In fact, part of the noise during the measurement process may indeed be Gaussian distributed, but there are other sources of noise during the experiment, such as walking on the surrounding ground, tapping on the table, moving objects on the table, etc., that are non-Gaussian distributed and which could increase the estimation error. Also, at the beginning of the training steps, the loss may risk divergence if the parameters are not set properly.





---

Another major difference is the dependence on the physical model, DLKF-FFS's performance depend heavily on how accurate the model is, whereas TCN does not require it at all. In contrast, TCN models require a large amount of high-quality training data. DLKF-FFS, on the other hand, does not require many training data sets.

---

## 6 Conclusion and Outlook

---

This thesis proposes two approaches for force estimation based on measurements from ferroelectret-based force sensors. First, a temporal convolutional neural network (TCN) is introduced as a causal, data-driven deep learning model. Second, we propose a hybrid approach, denoted as DLKF-FFS, which integrates a Kalman filter with a neural network. In this method, the Kalman filter treats the sensor measurement as the observation and updates its state using both the previous state and parameters predicted by a lightweight neural network, which itself takes the previous state as input.

Our first contribution lies in demonstrating that both models achieve accurate force estimation with low loss. Specifically, the TCN model performs well on short sequences, while DLKF-FFS achieves superior results on long sequences, with a 39.19% lower loss compared to TCN. Second, both models effectively mitigate non-ideal signal characteristics, reducing hysteresis by up to 44% and drift by up to 29%.

The second major finding is that the AI-aided Kalman filter is an efficient and fully interpretable method for processing two-channel time domain signals with high generalization and low number of parameters.

Another important observation is that the use of piezoelectric-based measurements provides more information for deep learning models, resulting in lower estimation errors compared to using only resistive strain gauge data.

The major limitation of this study is the single source of data since the measurement and experimentation is on only one type of sensor.

Future extensions of this work could involve expanding the Kalman filter's state vector to incorporate additional physical variables. Specifically, factors such as temperature, the piezoelectric coefficient  $d_{33}$ , and electrical resistivity  $\rho$  can be explicitly modeled. By integrating these variables, the model may better capture environmental and material dynamics, thereby improving its generalization ability.



---

More broadly, alternative variants of the Kalman filter can be explored to further enhance model performance and adaptability. In particular, the Extended Kalman Filter (EKF) may be employed to better capture system nonlinearities, offering a promising direction for improving estimation accuracy in complex scenarios.

---

## References

---

- [1] A. Matute, L. Paredes-Madrid, E. Gutierrez, and C. A. Parra Vargas, “Characterization of drift and hysteresis errors in force sensing resistors considering their piezocapacitive effect,” in *2017 IEEE SENSORS*, Glasgow: IEEE, Oct. 2017, pp. 1–3.
- [2] O. B. Dali, “Ferroelectrets: From material science to energy harvesting and sensor applications,” en, Ph.D. dissertation, Technische Universität Darmstadt, Darmstadt, Jun. 2023, pp. xvi, 149.
- [3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, Curran Associates, Inc., 2012.
- [4] A. Vaswani et al., “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17, Red Hook, NY, USA: Curran Associates Inc., Dec. 4, 2017, pp. 6000–6010.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.
- [6] J. Jumper et al., “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, no. 7873, pp. 583–589, Aug. 2021.
- [7] D. Silver et al., “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.
- [8] D. R. Sarvamangala and R. V. Kulkarni, “Convolutional neural networks in medical image understanding: A survey,” *Evolutionary Intelligence*, vol. 15, no. 1, pp. 1–22, Mar. 1, 2022.

- 
- 
- [9] F. A. Mohammed, K. K. Tune, B. G. Assefa, M. Jett, and S. Muhie, "Medical image classifications using convolutional neural networks: A survey of current methods and statistical modeling of the literature," *Machine Learning and Knowledge Extraction*, vol. 6, no. 1, pp. 699–735, Mar. 2024.
- [10] C. E. Petrillo et al., "Finding strong gravitational lenses in the kilo degree survey with convolutional neural networks," *Monthly Notices of the Royal Astronomical Society*, vol. 472, no. 1, pp. 1129–1150, Nov. 21, 2017.
- [11] N. Monsalves et al., "Application of convolutional neural networks to time domain astrophysics. 2d image analysis of OGLE light curves," *Astronomy & Astrophysics*, vol. 691, A106, Nov. 1, 2024.
- [12] A. Mondal, M. S. Manikandan, and R. B. Pachori, "Fast CNN-based electrocardiogram signal quality assessment using fourier magnitude spectrum for resource-constrained ECG diagnosis devices," *IEEE Sensors Letters*, vol. 8, no. 4, pp. 1–4, Apr. 2024.
- [13] Y. Gan, W. Li, Z. Zheng, J. Zhao, K. Zou, and B. Cai, "Cognitive radar signal processing based on convolutional neural network," in *2021 IEEE 6th International Conference on Signal and Image Processing (ICSIP)*, Nanjing, China: IEEE, Oct. 22, 2021, pp. 784–789.
- [14] L. Cheng, Y. Liu, J. Wei, H. Zhu, Z. Hu, and B. Luo, "Array-based faster r-CNN for underwater acoustic communication signal detection," in *2024 IEEE 8th International Conference on Vision, Image and Signal Processing (ICVISIP)*, Kunming, China: IEEE, Dec. 27, 2024, pp. 1–5.
- [15] O. Mohseni, J. Betz, B. Latsch, J. Seiler, A. Seyfarth, and M. Kupnik, "Ferroelectret-based insole for vertical ground reaction force estimation using a convolutional neural network," *IEEE Sensors Letters*, vol. 9, no. 5, pp. 1–4, May 2025.
- [16] C. Urrea and R. Agramonte, "Kalman filter: Historical overview and review of its use in robotics 60 years after its creation," *Journal of Sensors*, vol. 2021, no. 1, p. 9 674 015, 2021.
- [17] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint kalman filter for vision-aided inertial navigation," in *Proceedings 2007 IEEE International Conference on Robotics and Automation*, Apr. 2007, pp. 3565–3572.
- [18] T. Tran Phuong, C. Mitsantisuk, and K. Ohishi, "Multi-sensor fusion in kalman-filter for high performance force sensing," in *2011 IEEE International Conference on Industrial Technology*, Auburn, AL, USA: IEEE, Mar. 2011, pp. 393–398.

- 
- 
- [19] G. Puskorius and L. Feldkamp, "Neurocontrol of nonlinear dynamical systems with kalman filter trained recurrent networks," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 279–297, Mar. 1994.
- [20] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," *IEEE Signal Processing Magazine*, vol. 38, no. 2, pp. 18–44, Mar. 2021.
- [21] T. T. Phuong, K. Ohishi, and Y. Yokokura, "Deep learning based kalman filter for variable-frequency disturbance elimination in force sensing," in *2021 IEEE 30th International Symposium on Industrial Electronics (ISIE)*, Kyoto, Japan: IEEE, Jun. 20, 2021, pp. 1–6.
- [22] N. Shlezinger et al., "AI-aided kalman filters," *arXiv*, vol. arXiv/2410.12289, 2024.
- [23] J. Minase, T.-F. Lu, B. Cazzolato, and S. Grainger, "A review, supported by experimental results, of voltage, charge and capacitor insertion method for driving piezoelectric actuators," *Precision Engineering*, vol. 34, no. 4, pp. 692–700, Oct. 2010.
- [24] S. Kiranyaz, T. Ince, O. Abdeljaber, O. Avci, and M. Gabbouj, "1-d convolutional neural networks for signal processing applications," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, United Kingdom: IEEE, May 2019, pp. 8360–8364.
- [25] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv*, vol. arXiv/1803.01271, 2018.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770–778.
- [27] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37*, ser. ICML'15, Lille, France: JMLR.org, Jul. 6, 2015, pp. 448–456.
- [28] A. van den Oord et al., "WaveNet: A generative model for raw audio," in *Proc. 9th ISCA Workshop on Speech Synthesis Workshop*, 2016, pp. 125–125.